

JagozApp

Sistema Web de Gestão de Socios, Qoutas e bilhetica
com Controlo de Entradas por QR Code para Clube Desportivo
Local

Tomás Rocha
Tomás Pereira

Orientador: Luís Falcão

Relatório do projeto realizado no âmbito de Projecto e Seminário
Licenciatura em Engenharia Informática e de Computadores

Junho de 2026

INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA

JagozApp

49481 Tomás Rocha

49486 Tomás Correia Pereira

Orientador: Luís Falcão

Relatório do projeto realizado no âmbito de Projeto e Seminário
Licenciatura em Engenharia Informática e de Computadores

Junho de 2026

Resumo

O projeto Jagoz é uma aplicação web desenvolvida para apoiar a gestão do Grupo Desportivo União Ericeirense, centralizando processos administrativos ligados a sócios, atletas, equipas, patrocinadores, eventos, bilheteira, pagamentos e documentos.

A plataforma permite gerir inscrições de sócios e atletas, acompanhar estados de aprovação, organizar escalões e grupos de equipa, configurar opções de patrocínio, registar patrocinadores e controlar contratos de sponsorship. Inclui também funcionalidades para criação de eventos, venda de bilhetes, validação de dados de sócio, geração de recibos e integração com pagamentos online através da Stripe.

O sistema é composto por um backend em Kotlin com Spring Boot, responsável pela API REST, regras de negócio, autenticação, persistência e integrações externas, e por um frontend em React com Vite, que disponibiliza uma interface web para utilizadores, secretaria e administração. A persistência é feita em PostgreSQL, com acesso a dados através de JDBI.

A arquitetura está organizada por módulos, separando domínio, serviços, repositórios, camada HTTP e configuração da aplicação. Esta separação facilita a manutenção, os testes e a evolução futura do projeto. O projeto inclui ainda documentação OpenAPI, suporte Docker para execução local e testes automatizados para validar regras de domínio, mapeamentos HTTP e serviços principais.

Palavras-chave: aplicação web; sócios; atletas; patrocinadores ;React; Typescript; Spring Boot; PostgreSQL; Stripe; Vite; Docker

Abstract

The Jagoz project is a web application developed to support the management of Grupo Desportivo União Ericeirense, centralizing administrative processes related to members, athletes, teams, sponsors, events, ticketing, payments and documents.

The platform allows managing member and athlete registrations, tracking approval states, organizing age groups and team groups, configuring sponsorship options, registering sponsors and managing sponsorship contracts. It also includes features for creating events, selling tickets, validating member data, generating receipts and integrating with online payments through Stripe.

The system is composed of a backend written in Kotlin with Spring Boot, responsible for the REST API, business rules, authentication, persistence and external integrations, and a frontend written in React with Vite, which provides a web interface for users, secretariat and administration. Persistence is handled in PostgreSQL, with data access through JDBI.

The architecture is organized into modules, separating domain, services, repositories, the HTTP layer and the application configuration. This separation facilitates the maintenance, testing and future evolution of the project. The project also includes OpenAPI documentation, Docker support for local execution and automated tests to validate domain rules, HTTP mappings and the main services.

Keywords: web application; sports club management; members; athletes; sponsors; ticketing; Kotlin; Spring Boot; React; PostgreSQL; Stripe.

Utilização de Inteligência Artificial

Neste trabalho foram utilizadas ferramentas de inteligência artificial generativa, designadamente o Claude com intuito de auxiliar na criação de testes unitários e integrais para garantir todo o tipo de casos possíveis. Também foi usado na criação de ficheiros CSS, com o objetivo de ter uma UI moderna. Por fim, esta ferramenta de inteligência artificial também foi importante na discussão de implementações e integrações relacionadas com o projeto, como por exemplo, qual é a melhor ferramenta de pagamento e como esta pode ser integrada seguramente no projeto.

Índice

1	Introdução	1
1.1	Motivação	1
1.2	Objetivos	1
1.3	Terminologia	2
2	Enquadramento	3
2.1	Gestão de Clubes Desportivos de Pequena Dimensão	3
2.2	Sistemas Semelhantes	3
3	Requisitos e Arquitetura da Aplicação	5
3.1	Visão Geral da Aplicação	5
3.2	Requisitos Funcionais	6
3.2.1	Requisitos Obrigatórios	6
3.2.2	Requisitos Complementares	7
3.3	Requisitos Não Funcionais	7
3.4	Arquitetura Geral	8
3.5	Arquitetura Modular	8
3.6	Principais Fluxos da Aplicação	10
3.6.1	Fluxo de Autenticação	10
3.6.2	Fluxo de Inscrição de Sócio	11
3.6.3	Fluxo de Inscrição de Atleta	11
3.6.4	Fluxo de Patrocínio	12
3.6.5	Fluxo de Compra de Bilhetes	12
3.7	Síntese	13
4	Backend	15
4.1	Estrutura	15
4.2	Repositórios e Base de Dados	17
4.2.1	Repositório de Dados	17
4.2.2	Repositório de Ficheiros	21
4.3	Serviços	22

4.4	Controladores	23
4.5	Autenticação e Autorização	24
4.6	Integrações Externas	25
4.6.1	Pagamentos com Stripe	25
4.6.2	Envio de Correio Eletrónico	25
4.6.3	Armazenamento de Ficheiros	26
4.6.4	Geração de Códigos QR	26
5	Frontend	27
5.1	Estrutura da Aplicação	27
5.2	Navegação e Autenticação	28
5.3	Interface da Aplicação	28
5.4	Comunicação com a API	29
5.5	Gestão de Estado e Hooks	29
5.6	Componentes Reutilizáveis	30
5.7	Rotas da Aplicação	31
5.8	Funcionalidades por Domínio	32
5.8.1	Página Inicial	32
5.8.2	Autenticação	33
5.8.3	Sócios	33
5.8.4	Atletas	34
5.8.5	Patrocínios	35
5.8.6	Eventos e Bilheteira	36
5.8.7	Utilizadores e Perfil	36
5.8.8	Ficheiros	37
5.8.9	Administração	37
5.9	Formulários e Validação	38
5.10	Tipos e Segurança em Tempo de Desenvolvimento	38
5.11	Tratamento de Estados da Interface	38
5.12	Estilo Visual e Responsividade	39
5.13	Internacionalização	39
6	Conclusão	41
	Referências	43
A	Exemplo de apêndice	45

Lista de Figuras

3.1	Arquitetura geral da aplicação	8
3.2	Dependências entre os módulos do backend	10
3.3	Fluxo de autenticação	11
3.4	Fluxo de adesão de sócio	11
3.5	Fluxo de inscrição de atleta	12
3.6	Fluxo de submissão e aprovação de patrocínio	12
3.7	Fluxo de compra de bilhetes e confirmação de pagamento	13
4.1	Modelo de dados – utilizadores, sócios e atletas	19
4.2	Modelo de dados – equipas e tabelas de preços	20
4.3	Modelo de dados – patrocínios	20
4.4	Modelo de dados – cobranças, pagamentos e eventos	21
5.1	Página inicial da aplicação	32
5.2	Página de autenticação	33
5.3	Formulário de adesão de sócio	34
5.4	Página de perfil de atleta	35
5.5	Página de Informação sobre Patrocínios	36
5.6	Página Inicial do BackOffice	37

Capítulo 1

Introdução

Este capítulo tem como objetivo apresentar a motivação do projeto e as necessidades que este irá preencher.

1.1 Motivação

Durante a fase de definição do projeto, surgiu uma proposta por parte do presidente do Grupo Desportivo União Ericeirense, dirigida aos autores deste trabalho, então a concluir a Licenciatura em Engenharia Informática e de Computadores. Durante a apresentação da proposta, tornou-se evidente a existência de uma necessidade concreta: reforçar a ligação entre o clube e os adeptos.

A ausência de uma plataforma digital representava, para o clube, a perda de diversas oportunidades. Muitos potenciais sócios e patrocinadores acabavam por não concretizar a sua adesão devido à reduzida disponibilidade para se deslocarem presencialmente à sede do clube. Identificou-se, assim, uma oportunidade clara de intervenção, capaz de colmatar estas dificuldades.

A solução inicialmente equacionada passava pelo desenvolvimento de uma aplicação móvel. Contudo, a burocracia e os requisitos de licenciamento associados à publicação em plataformas como a App Store e a Google Play levaram à adoção de uma alternativa mais acessível: uma aplicação web. Esta abordagem permite disponibilizar as funcionalidades pretendidas sem as restrições inerentes à distribuição em lojas de aplicações móveis.

1.2 Objetivos

O objetivo central deste projeto é desenvolver uma web app que reforce a ligação entre o Grupo Desportivo União Ericeirense e a sua comunidade, disponibilizando online um conjunto de processos que tradicionalmente exigiam deslocação presencial à sede do clube.

Para concretizar este propósito, definiram-se os 4 objetivos, sendo o último o que os dirigentes do clube consideraram menos crucial, no entanto enriquecia a complexidade do projeto:

- Permitir que qualquer pessoa se torne sócio do clube através da aplicação web;
- Permitir a adesão de novos patrocinadores de forma totalmente online;
- Disponibilizar a inscrição de atletas no clube;
- Implementar a venda de bilhetes para os jogos, com validação por código QR.

1.3 Terminologia

Esta secção é dedicada à definição de termos usados ao longo deste documento.

- *frontend* – interface, parte da aplicação que é executada no navegador do utilizador;
- *backend* – componente da aplicação que é executada no servidor;
- *backoffice* – área de gestão interna utilizada pelo clube;
- *role* – nível de permissão atribuído a um utilizador;
- *check-in* – validação da entrada através da leitura do código QR;
- *id* – identificador único;
- *QR Code* – *Quick Response Code*, código de barras bidimensional;

Capítulo 2

Enquadramento

Este capítulo está organizado em duas secções. A primeira descreve o contexto da gestão de clubes desportivos de menor dimensão e a realidade específica do clube para o qual a aplicação foi desenvolvida. A segunda apresenta soluções existentes neste domínio e posiciona a aplicação JagozApp face a essas soluções.

2.1 Gestão de Clubes Desportivos de Pequena Dimensão

A gestão de um clube desportivo de pequena ou média dimensão envolve um conjunto alargado de tarefas administrativas, financeiras e de comunicação que, com frequência, são asseguradas por um número reduzido de pessoas, muitas vezes em regime de voluntariado. Processos como a gestão de sócios, a inscrição de atletas, o controlo de pagamentos ou a angariação de patrocínios tendem a estar dispersos por folhas de cálculo, formulários em papel, correio eletrónico e comunicações informais.

No caso concreto do Grupo Desportivo União Ericeirense, identificou-se uma necessidade que ultrapassa a gestão interna: a aproximação entre o clube e a sua comunidade. A ausência de meios digitais orientados para o exterior traduzia-se em oportunidades perdidas, tanto na angariação de novos sócios como de patrocinadores, sobretudo junto de pessoas com pouca disponibilidade para se deslocarem presencialmente à sede do clube.

2.2 Sistemas Semelhantes

Existem diferentes tipos de soluções relacionadas com a presença digital de um clube desportivo, que podem ser agrupadas em duas categorias.

A primeira categoria corresponde às **plataformas de gestão de clubes**, das quais o EMJOGO [1] é um exemplo representativo no contexto português. Trata-se de uma solução comercial, disponibilizada no modelo de *software* como serviço (SaaS), que centraliza numa única plataforma a gestão desportiva, administrativa e financeira do clube, incluindo a gestão de atletas, sócios, mensalidades e patrocinadores. O EMJOGO disponibiliza ainda um *website* e uma aplicação de comunicação com a comunidade do clube.

A segunda categoria corresponde à **presença *online* própria de cada clube**, que, no caso dos clubes distritais de menor dimensão, se traduz maioritariamente em *websites* de carácter institucional e informativo. Estes apresentam a história do clube, as suas equipas e notícias, mas raramente disponibilizam funcionalidades interativas que permitam ao adepto realizar ações diretamente, como aderir como sócio, inscrever um atleta ou adquirir bilhetes.

É neste contexto que o clube apresentou a sua proposta inicial. O Grupo Desportivo União Ericeirense já utiliza o EMJOGO para a sua gestão interna, e o objetivo inicialmente colocado era o de estabelecer uma ligação entre a nova aplicação e essa plataforma, de forma a evitar a duplicação manual de dados. Contudo, o EMJOGO é uma solução comercial de código fechado e não disponibiliza publicamente uma interface de integração que o permitisse. Por esse motivo, uma integração direta não se revelou viável no âmbito deste projeto.

Perante esta limitação, o foco do projeto foi redefinido. Em vez de procurar centralizar ou integrar a gestão já assegurada pelo EMJOGO, a aplicação JagozApp passou a ter como objetivo complementar essa plataforma, oferecendo ao clube aquilo que considera ser a sua principal lacuna no contexto específico de um clube de pequena dimensão: uma ligação direta e acessível entre o clube e o adepto. Assim, a JagozApp distingue-se por reunir, numa aplicação *web* desenvolvida à medida do clube, um conjunto de funcionalidades voltadas para o exterior — a adesão de sócios, a inscrição de atletas, o registo de patrocinadores e a venda de bilhetes com validação por código QR. Ao contrário de uma solução comercial genérica, esta é uma aplicação própria do clube, sobre a qual este detém controlo total e que pode evoluir de acordo com as suas necessidades futuras.

Capítulo 3

Requisitos e Arquitetura da Aplicação

Este capítulo apresenta a visão funcional e arquitetural da JagozApp. Em primeiro lugar são descritos os requisitos funcionais e não funcionais do projeto, distinguindo entre funcionalidades obrigatórias e funcionalidades complementares. De seguida é apresentada a arquitetura geral da aplicação, os principais fluxos de utilização e as decisões arquiteturais que orientaram a implementação.

3.1 Visão Geral da Aplicação

A JagozApp foi desenvolvida com o objetivo de centralizar a gestão operacional de um clube desportivo local, reduzindo a dependência de processos manuais, ficheiros dispersos e comunicação informal. A aplicação cobre várias áreas de funcionamento do clube: gestão de sócios, inscrições de atletas, patrocínios, eventos, bilhética, pagamentos, documentos e administração interna.

Do ponto de vista dos utilizadores, a aplicação distingue três perfis principais:

- **Visitante:** utilizador não autenticado, com acesso a páginas públicas, informação institucional, consulta de atletas por escalão, informação de patrocínios e compra de bilhetes;
- **Utilizador autenticado:** utilizador com conta, que pode gerir o seu perfil, submeter pedidos, inscrever atletas, consultar patrocínios associados e realizar pagamentos;
- **Administração/Secretaria:** utilizador com permissões de gestão, responsável por aprovar pedidos, gerir sócios, atletas, patrocinadores, eventos, bilhetes, utilizadores e configurações.

A aplicação foi concebida como um sistema web completo, composto por uma interface *frontend*, uma API *backend*, uma base de dados relacional e integrações externas para pagamentos, correio eletrónico e armazenamento de ficheiros.

3.2 Requisitos Funcionais

Os requisitos funcionais representam as funcionalidades que o sistema deve disponibilizar aos seus utilizadores. Estes requisitos foram organizados por domínio funcional.

3.2.1 Requisitos Obrigatórios

Os requisitos obrigatórios correspondem às funcionalidades essenciais para que a aplicação cumpra o seu objetivo principal.

- **Autenticação e gestão de utilizadores:** permitir o registo, início de sessão, fim de sessão e identificação do utilizador autenticado;
- **Autorização por papéis:** distinguir permissões entre utilizadores normais, secretaria e administradores;
- **Gestão de sócios:** permitir criar, consultar, editar, aprovar, rejeitar, reativar e desativar sócios;
- **Inscrição e gestão de atletas:** permitir a inscrição de atletas, incluindo dados pessoais, desportivos, escolares e encarregados de educação;
- **Consulta pública de atletas:** disponibilizar informação não sensível sobre atletas por escalão;
- **Gestão de equipas e escalões:** permitir configurar grupos, escalões e respetiva organização;
- **Gestão de patrocinadores:** permitir criar e consultar patrocinadores;
- **Gestão de patrocínios:** permitir submeter, aprovar, cancelar e marcar patrocínios como pagos;
- **Catálogo de patrocínios:** permitir configurar opções de publicidade, patrocínio de equipas e outros apoios;
- **Gestão de eventos:** permitir criar, consultar, editar e cancelar eventos;
- **Bilhética:** permitir a compra de bilhetes para eventos e a validação de condições de sócio quando aplicável;
- **Pagamentos:** permitir iniciar pagamentos online e registar o seu estado;
- **Recibos:** permitir consultar recibos de pagamentos e de patrocínios;
- **Gestão de ficheiros:** permitir carregar, consultar e remover documentos ou fotografias associados a entidades da aplicação;

- **Painel administrativo:** apresentar indicadores gerais sobre o estado do clube e acesso às áreas de gestão.

3.2.2 Requisitos Complementares

Para além das funcionalidades essenciais, foram identificados requisitos complementares que aumentam a qualidade da solução e melhoram a experiência de utilização.

- **Internacionalização:** disponibilizar a interface em português e inglês;
- **Execução em Docker:** permitir correr a aplicação localmente através de containers;
- **Documentação da API:** disponibilizar uma especificação OpenAPI para consulta dos *endpoints*;
- **Testes automatizados:** validar regras de domínio, mapeamentos HTTP e serviços principais;
- **Notificações por email:** enviar confirmações associadas a fluxos como a compra de bilhetes;
- **Geração de códigos QR:** associar bilhetes confirmados a códigos QR para posterior validação;
- **Armazenamento externo de ficheiros:** guardar conteúdo binário fora da base de dados relacional.

3.3 Requisitos Não Funcionais

Os requisitos não funcionais definem propriedades de qualidade que a aplicação deve respeitar.

- **Manutenibilidade:** a aplicação deve estar organizada por módulos e camadas, facilitando alterações futuras;
- **Testabilidade:** a lógica de domínio e de serviços deve poder ser testada de forma isolada;
- **Segurança:** palavras-passe não devem ser guardadas em texto simples e o acesso a dados sensíveis deve exigir autenticação e autorização;
- **Privacidade:** dados sensíveis de sócios e atletas não devem ser expostos em páginas públicas;
- **Consistência:** operações que envolvem múltiplas entidades devem ser executadas dentro de transações;

- **Usabilidade:** a interface deve ser clara, organizada e adequada a utilizadores com diferentes níveis de conhecimento técnico;
- **Portabilidade:** o projeto deve poder ser executado em ambiente local através de Docker;
- **Escalabilidade evolutiva:** a arquitetura deve permitir a adição de novas funcionalidades sem reestruturações profundas.

3.4 Arquitetura Geral

A arquitetura da JagozApp segue uma separação clara entre cliente, servidor, persistência e serviços externos. O *frontend*, implementado em React, é responsável pela interação com o utilizador. O *backend*, implementado em Kotlin com Spring Boot, expõe uma Web API REST e concentra a lógica de negócio. A persistência dos dados estruturados é feita em PostgreSQL, enquanto ficheiros, pagamentos e email são delegados a serviços externos.

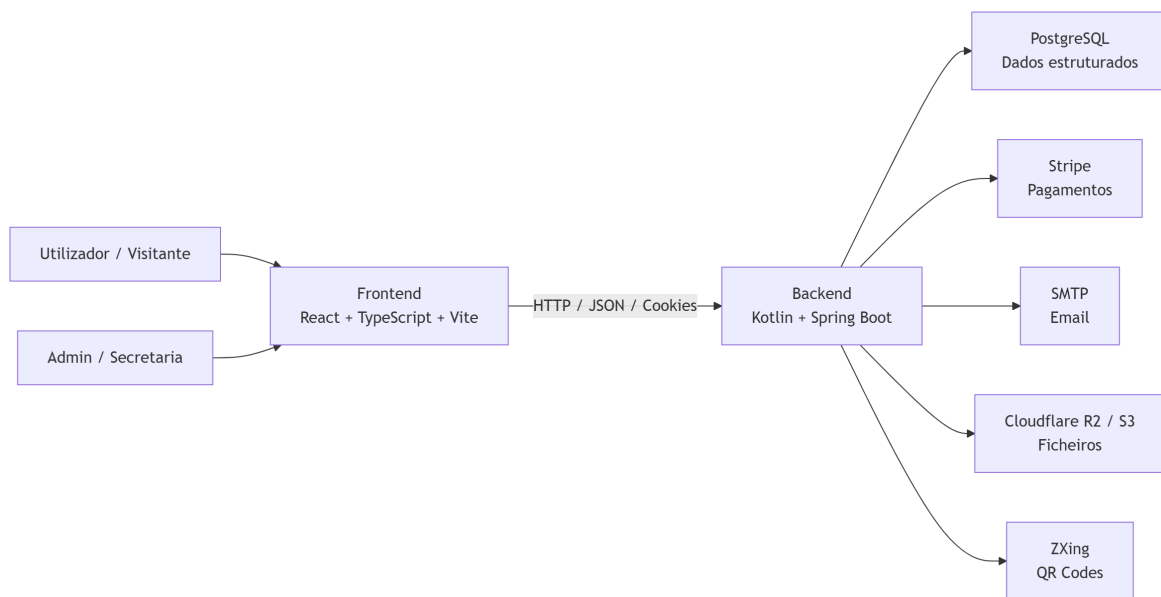


Figura 3.1: Arquitetura geral da aplicação

Esta divisão permite separar responsabilidades e reduzir o acoplamento entre componentes. O *frontend* não acede diretamente à base de dados, fazendo todos os pedidos através da API. O *backend*, por sua vez, concentra a validação das regras de negócio e controla o acesso aos dados e integrações.

3.5 Arquitetura Modular

O projeto foi organizado como uma aplicação multi-módulo. Esta decisão foi tomada para separar claramente as responsabilidades internas do sistema e evitar que todas as classes fi-

cassem concentradas num único módulo. Embora esta separação seja mais visível no *backend*, ela influencia a forma como a aplicação é mantida e testada.

A organização em módulos permite:

- isolar a lógica de domínio de detalhes tecnológicos;
- impedir que os controladores dependam diretamente da base de dados;
- substituir implementações de repositórios sem alterar os serviços;
- testar regras de negócio sem iniciar a aplicação completa;
- tornar explícitas as dependências entre camadas;
- facilitar a evolução futura do projeto.

Os principais módulos do *backend* são:

- **domain**: entidades, enumerações, validações e regras puras de domínio;
- **repository**: contratos de acesso a dados;
- **repository-jdbi**: implementação concreta dos repositórios em PostgreSQL através de JDBI;
- **services**: casos de uso e lógica de negócio da aplicação;
- **http**: controladores REST, modelos HTTP e pipeline de autenticação;
- **host**: módulo de arranque e configuração da aplicação Spring Boot.

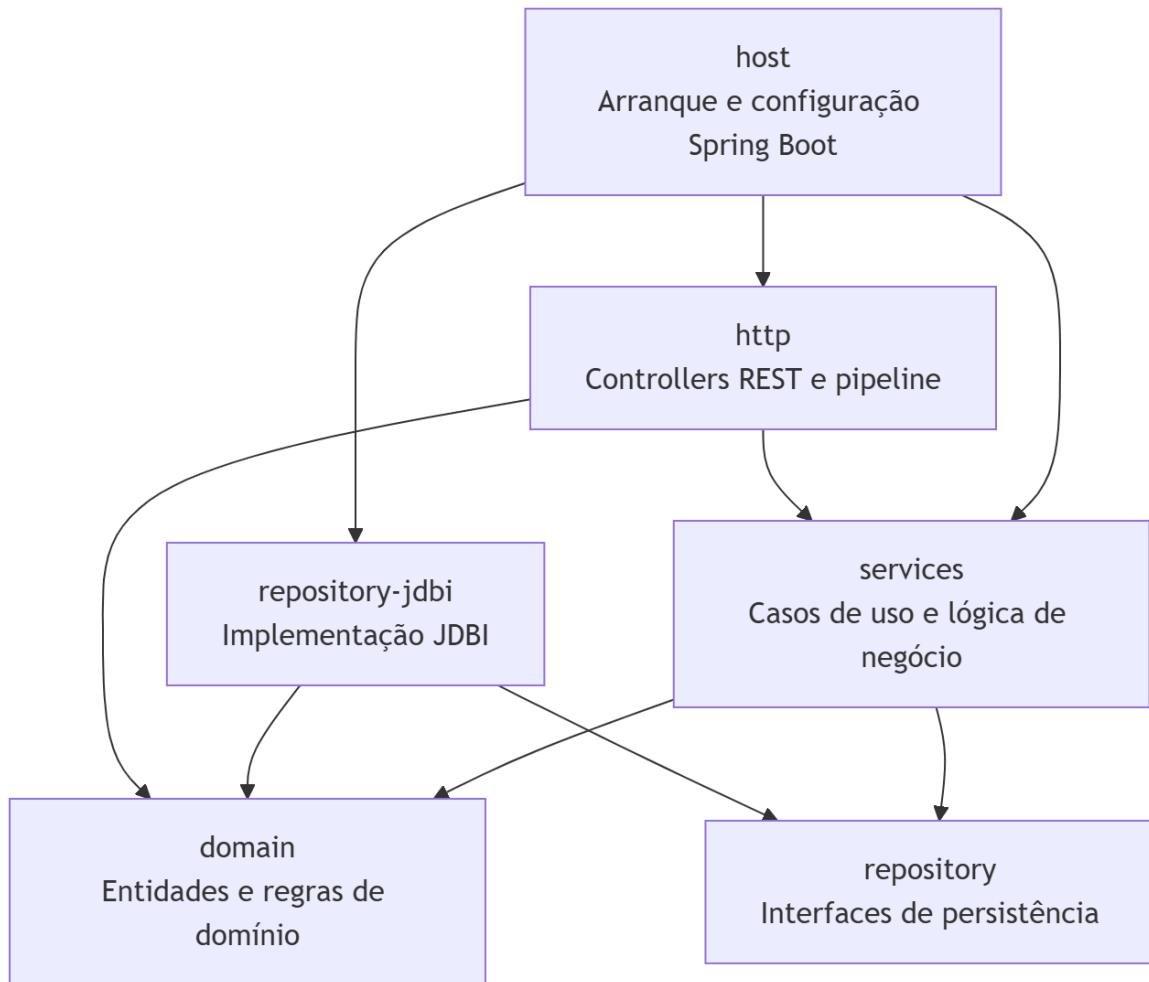


Figura 3.2: Dependências entre os módulos do backend

3.6 Principais Fluxos da Aplicação

Esta secção descreve os principais fluxos de utilização suportados pela aplicação.

3.6.1 Fluxo de Autenticação

O fluxo de autenticação começa quando o utilizador submete as suas credenciais no *frontend*. O *backend* valida a palavra-passe e, em caso de sucesso, cria um *token* de sessão. Esse *token* é enviado ao cliente através de um *cookie* e usado nos pedidos seguintes.

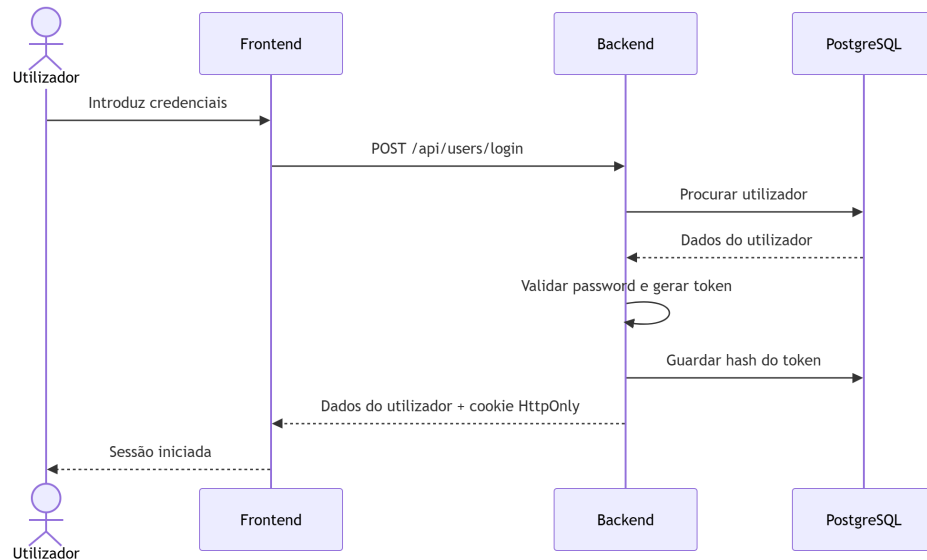


Figura 3.3: Fluxo de autenticação

3.6.2 Fluxo de Inscrição de Sócio

A adesão de sócio é um dos fluxos centrais da aplicação. O utilizador preenche um formulário no *frontend*, que envia os dados para a API. O *backend* valida os dados, cria o registo de sócio em estado pendente e disponibiliza-o para análise pela secretaria ou administração.

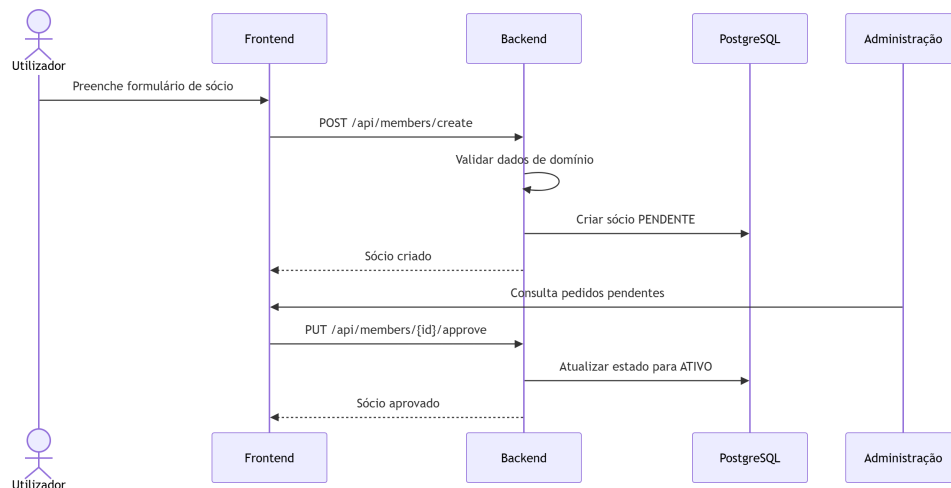


Figura 3.4: Fluxo de adesão de sócio

3.6.3 Fluxo de Inscrição de Atleta

A inscrição de atleta é mais complexa, uma vez que envolve dados pessoais, dados desportivos e encarregados de educação. No *backend*, esta operação cria, numa única transação, o sócio associado, o atleta e os respetivos encarregados. Esta abordagem garante que não existem atletas incompletos caso alguma parte da operação falhe.

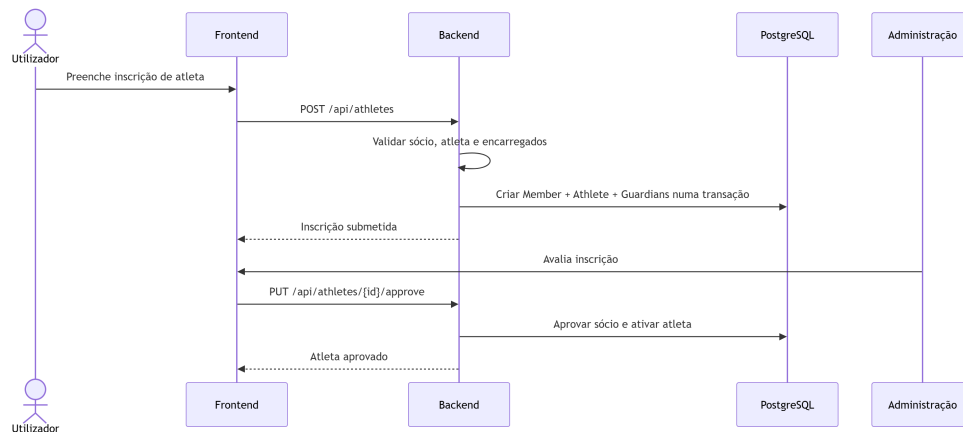


Figura 3.5: Fluxo de inscrição de atleta

3.6.4 Fluxo de Patrocínio

No fluxo de patrocínio, o utilizador escolhe uma opção disponível, preenche os dados do patrocinador e submete o pedido. A secretaria ou administração pode posteriormente aprovar o pedido, marcar o pagamento e acompanhar o estado do contrato.

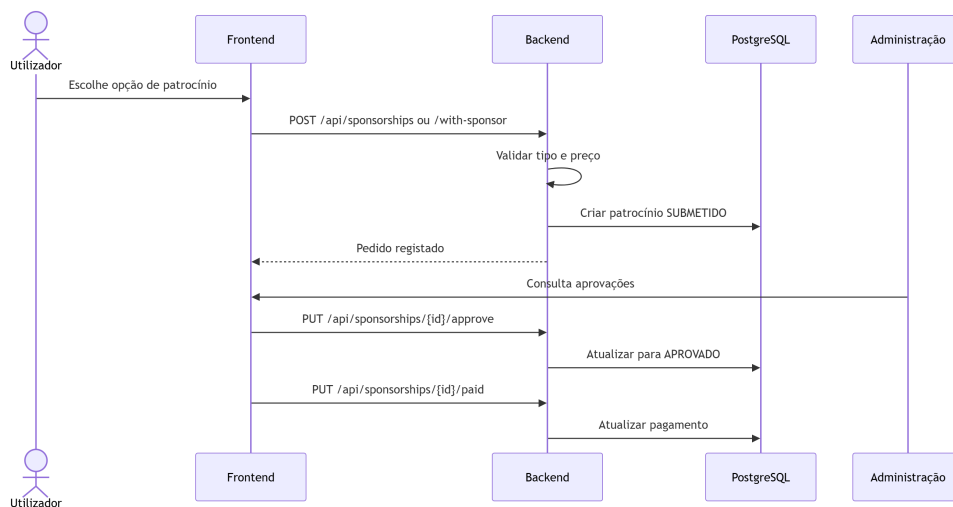


Figura 3.6: Fluxo de submissão e aprovação de patrocínio

3.6.5 Fluxo de Compra de Bilhetes

Na compra de bilhetes, o utilizador seleciona um evento e o setor pretendido. Quando o bilhete é de sócio, a aplicação valida o número de sócio e a data de nascimento. Depois é criada uma sessão de pagamento na Stripe. Quando o pagamento é confirmado, o *backend* confirma os bilhetes, gera os códigos QR e envia a confirmação por email.

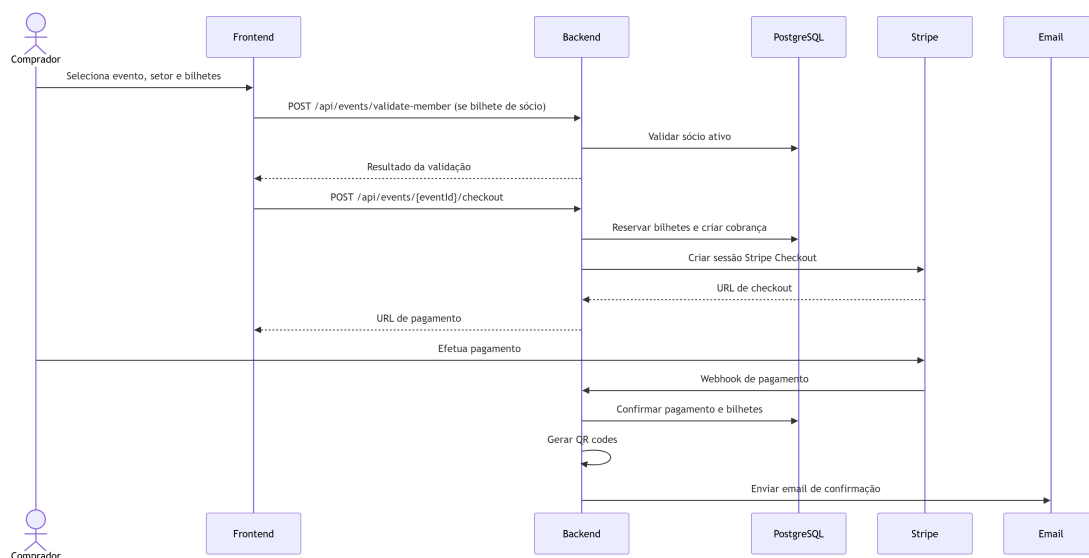


Figura 3.7: Fluxo de compra de bilhetes e confirmação de pagamento

3.7 Síntese

A JagozApp foi desenhada para responder a um conjunto alargado de necessidades de gestão de um clube desportivo local. A separação entre *frontend*, *backend*, base de dados e integrações externas permite organizar o sistema de forma clara. A adoção de uma arquitetura multi-módulo no *backend* reforça esta separação, promovendo manutenção, testabilidade e evolução futura.

Os requisitos definidos cobrem os processos essenciais do clube, desde a gestão de sócios e atletas até à bilhética, pagamentos, patrocínios e documentos. Os diagramas apresentados ajudam a compreender como estes processos atravessam as várias camadas da aplicação e como cada componente contribui para o funcionamento global do sistema.

Capítulo 4

Backend

Este capítulo está organizado em seis secções. A primeira é dedicada à estrutura geral da aplicação; a segunda foca-se nos repositórios e no modelo de dados; a terceira é referente aos serviços; a quarta trata dos controladores; a quinta é focada no processo de autenticação e autorização; e por fim, a sexta secção descreve as integrações com serviços externos.

O backend da aplicação JagozApp fornece uma interface de comunicação entre o *frontend* e a lógica central da aplicação através de uma Web API. Uma Web API (*Application Programming Interface*) é, no fundo, um conjunto de *endpoints* expostos sobre o protocolo HTTP (*Hypertext Transfer Protocol*), que permite a diferentes aplicações comunicarem entre si.

O backend foi implementado na linguagem Kotlin [2], utilizando a framework Spring Boot [3], que oferece um conjunto de abstrações e configurações que aceleram o desenvolvimento e promovem boas práticas. Esta framework permite a construção de APIs REST de forma estruturada e eficiente. Uma das principais vantagens do Spring é o suporte nativo a conceitos fundamentais como:

- Inversão de controlo (IoC): permite delegar à framework a responsabilidade de instanciar e gerir o ciclo de vida de objetos, facilitando o desacoplamento entre componentes;
- Injeção de dependências (DI): injeta automaticamente as dependências necessárias nos componentes da aplicação;
- Separação de responsabilidades: através da arquitetura em camadas (*controllers*, *services*, *repositories*), incentiva a modularidade e organização do código.

4.1 Estrutura

O backend da aplicação foi desenvolvido segundo uma arquitetura modular e orientada a serviços, com uma clara separação de responsabilidades entre as diferentes camadas. Esta abordagem facilita a manutenção, a organização, a realização de testes e a extensibilidade do sistema.

A organização do código está dividida em vários módulos, cada um com uma responsabilidade bem definida:

- **domain:** contém as entidades, enumerações e a lógica de domínio, independente de qualquer tecnologia externa;
- **services:** contém a lógica de negócio da aplicação, organizada em casos de uso;
- **repository:** define as interfaces de acesso a dados, sem qualquer dependência da tecnologia de persistência utilizada;
- **repository-jdbi:** contém a implementação concreta das interfaces de repositório, recorrendo à biblioteca JDBI;
- **http:** contém os controladores REST e o respetivo *pipeline* de processamento de pedidos;
- **host:** módulo de arranque da aplicação Spring Boot, responsável pela configuração e pela ligação entre todos os módulos.

A arquitetura segue o padrão comum de três camadas: controladores, serviços e repositórios. Adicionalmente, são utilizadas interfaces para abstrair o acesso a dados, o que permite alternar facilmente entre diferentes implementações (por exemplo, uma implementação em memória para testes e uma implementação JDBI para produção).

A seguir são descritos os principais componentes:

- *Controllers* ou controladores: são responsáveis por receber e tratar os pedidos HTTP provenientes do *frontend*. Traduzem os pedidos da camada de apresentação para chamadas aos serviços da aplicação e retornam as respostas apropriadas;
- *Services* ou serviços: contêm a lógica de negócio da aplicação. São responsáveis por coordenar as ações necessárias à execução de cada operação;
- *Repositories* ou repositórios: abstraem o acesso aos dados persistentes. O acesso a dados estruturados é feito através da base de dados relacional, enquanto o armazenamento de ficheiros é delegado a um serviço de armazenamento dedicado.

Importa ainda destacar duas abstrações transversais a toda a aplicação. A primeira é o tipo `Either<L, R>`, que encapsula o sucesso (`Right`) ou a falha (`Left`) de uma operação. Os serviços seguem esta abordagem de retorno funcional, o que torna o tratamento de erros explícito e evita o recurso a exceções não controladas. A segunda é o `TransactionManager`, que permite que cada operação de um serviço seja executada no contexto de uma transação, garantindo a consistência das operações que envolvem múltiplos repositórios.

4.2 Repositórios e Base de Dados

Neste projeto, verificou-se a necessidade de uma divisão em dois tipos de repositórios, cada um associado a uma forma de armazenamento distinta:

- Repositório de dados – responsável por armazenar e consultar dados estruturados, como utilizadores, sócios, atletas, patrocínios, pagamentos e eventos, através de uma base de dados relacional;
- Repositório de ficheiros – responsável por armazenar os ficheiros associados às entidades, como fotografias e documentos, através de um serviço de armazenamento externo.

Ambos os repositórios são abstraídos através de interfaces, o que permite a substituição ou simulação de comportamentos para testes. As interfaces estão definidas no módulo `repository`, enquanto a sua implementação concreta reside no módulo `repository-jdbi`. Desta forma, as camadas de serviços e de controladores nunca dependem diretamente da tecnologia de persistência utilizada.

4.2.1 Repositório de Dados

Este repositório utiliza a base de dados relacional PostgreSQL [4], escolhida pelas seguintes razões:

- Familiaridade;
- Documentação extensiva;
- Desempenho estável e fiável para dados relacionais;
- Facilidade de integração com bibliotecas Kotlin como o JDBI.

O acesso à base de dados é feito através da biblioteca JDBI [5], que permite executar *queries* parametrizadas em SQL nativo com uma boa integração em Kotlin, mantendo o controlo explícito sobre as operações realizadas. Ao contrário de soluções baseadas em ORM (Object-Relational Mapping), esta abordagem evita camadas de abstração adicionais e mantém o SQL visível e sob controlo do programador.

Base de dados

O modelo de dados está organizado em torno de várias áreas funcionais da aplicação. As principais entidades são descritas a seguir.

A entidade `User` representa um utilizador da aplicação e armazena o identificador, o *email*, o nome de utilizador, a informação de validação da palavra-passe e o papel (`role`) atribuído. Cada utilizador pode estar opcionalmente associado a um sócio ativo. A informação de

autenticação é complementada pela entidade **Token**, que representa as sessões ativas de cada utilizador.

A entidade **Member** corresponde a um sócio do clube e concentra os seus dados pessoais e administrativos, como o número de sócio, o nome completo, a data de nascimento, os contactos, a morada, o NIF, a categoria, o estado e a quota associada. A entidade **Athlete** representa um atleta e está sempre associada a um sócio, numa relação de um para um. Um atleta contém informação desportiva e escolar, bem como uma lista de encarregados de educação, representados pela entidade **Guardian**.

A organização das equipas é feita através das entidades **TeamGroup** (grupo ou modalidade) e **TeamCategory** (escalão), às quais os atletas são associados. A estas juntam-se entidades dedicadas à definição das tabelas de preços.

A área de patrocínios é composta pela entidade **Sponsor**, que representa um patrocinador, e pela entidade **Sponsorship**, que representa um contrato de patrocínio. Existem ainda entidades de catálogo que definem as opções de patrocínio disponíveis.

A componente financeira assenta nas entidades **Charge** (cobrança), **ChargeItem** (item de uma cobrança) e **Payment** (pagamento). Uma cobrança pode estar associada a um sócio ou a um contrato de patrocínio, e dá origem a um ou mais pagamentos.

Por fim, a área de eventos é composta pela entidade **Event** (jogo), pela entidade **EventSector** (setor com lotação) e pela entidade **Ticket** (bilhete), que suporta a funcionalidade relacionada com os bilhetes.

Vários campos, como o papel de um utilizador, o estado de um sócio ou o estado de um contrato de patrocínio, são restritos a um conjunto de valores predefinidos, implementados como enumerações para garantir a consistência e a integridade dos dados.

Dado o número de entidades envolvidas, o modelo de dados é apresentado em quatro diagramas, organizados por área funcional. A Figura 4.1 representa o núcleo de utilizadores, sócios e atletas; a Figura 4.2 representa a organização das equipas e as tabelas de preços; a Figura 4.3 representa a área de patrocínios; e a Figura 4.4 representa as áreas financeira e de eventos. As enumerações associadas a cada área estão representadas no respetivo diagrama.

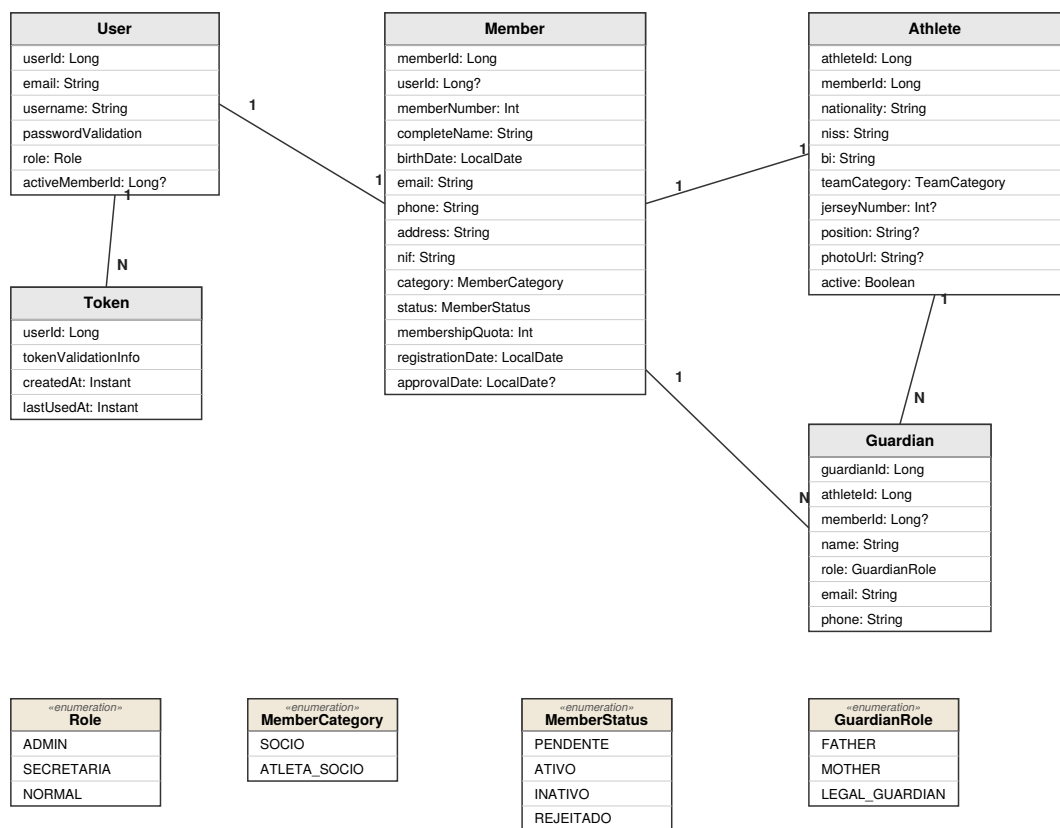


Figura 4.1: Modelo de dados – utilizadores, sócios e atletas

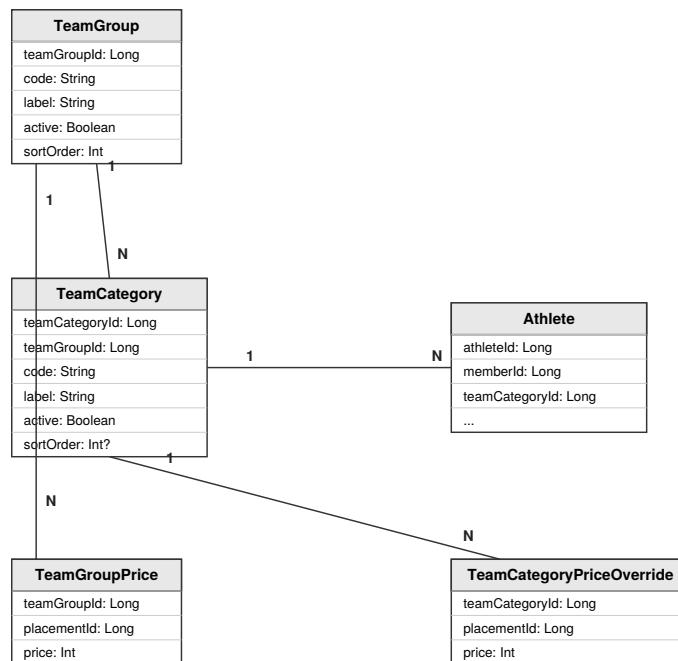


Figura 4.2: Modelo de dados – equipas e tabelas de preços

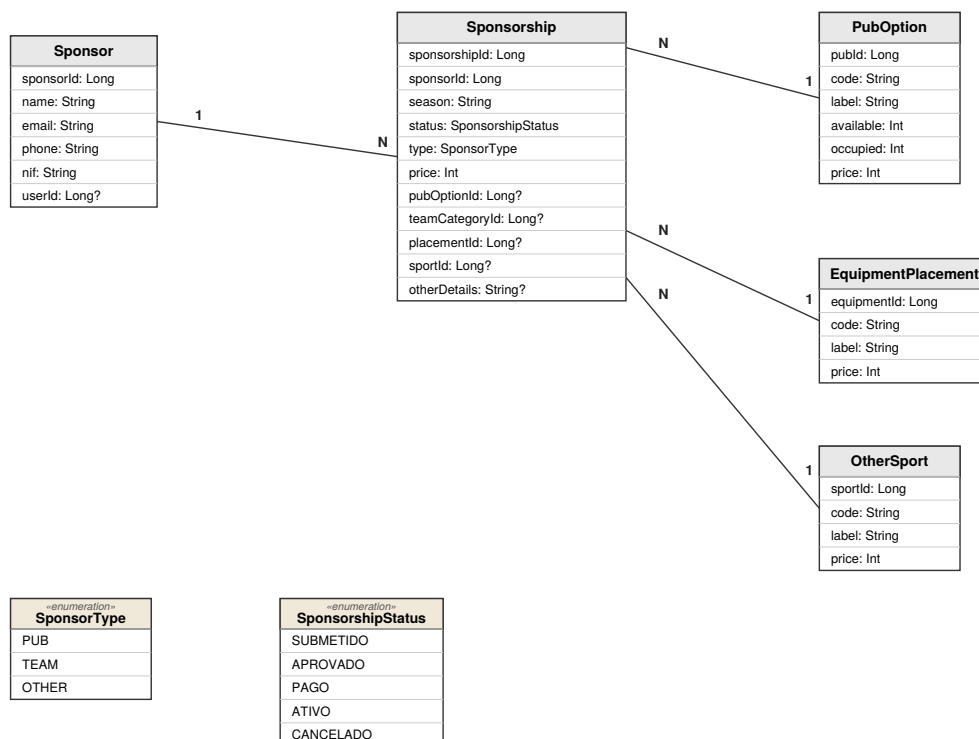


Figura 4.3: Modelo de dados – patrocínios

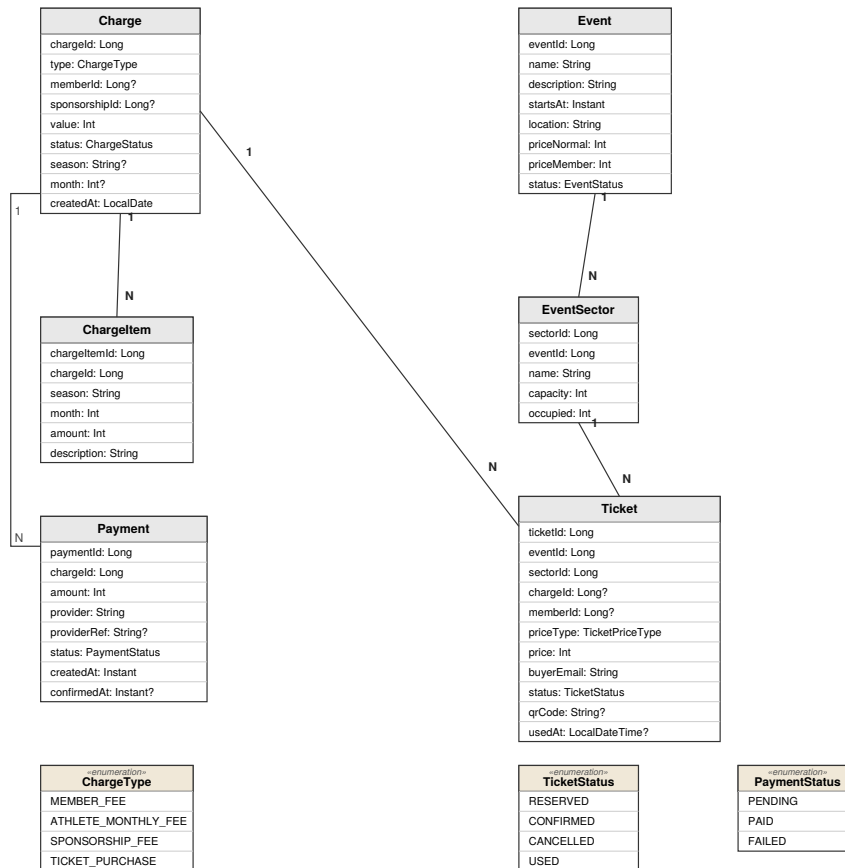


Figura 4.4: Modelo de dados – cobranças, pagamentos e eventos

4.2.2 Repositório de Ficheiros

O repositório de ficheiros é responsável pelo armazenamento de recursos binários associados às entidades da aplicação, nomeadamente fotografias de perfil, fotografias de sócios e atletas, e documentos como o cartão de cidadão ou o exame médico dos atletas.

Importa distinguir dois tipos de informação. Os *metadados* de cada ficheiro – como o nome original, o tipo de conteúdo, o tamanho e a chave de armazenamento – são guardados na base de dados relacional, na entidade **StoredFile**. Já o conteúdo binário dos ficheiros não é guardado na base de dados, sendo delegado a um serviço de armazenamento externo, descrito em detalhe na Secção 4.6.

Cada ficheiro é identificado por uma chave de armazenamento construída de forma hierárquica, que combina o tipo e o identificador da entidade proprietária, o tipo de ficheiro e um identificador único. É ainda aplicada uma política de um único ficheiro ativo por entidade e tipo, o que significa que o carregamento de um novo ficheiro substitui o anterior do mesmo tipo. O carregamento de ficheiros está sujeito a validações de tipo e tamanho, distintas consoante

se trate de imagens ou de documentos.

4.3 Serviços

A camada de serviços é responsável pela lógica de negócio da aplicação. É nela que se realiza o processamento dos dados, a aplicação de regras e a orquestração das chamadas aos repositórios e aos serviços externos. Cada serviço é anotado com `@Named` e recebe as suas dependências por injeção no construtor, e recorre ao `TransactionManager` para que as operações que envolvem múltiplos repositórios sejam executadas no contexto de uma única transação.

Todos os serviços seguem uma abordagem de retorno funcional baseada no tipo genérico `Either<L, R>`, que encapsula o sucesso (`Right`) ou a falha (`Left`) de uma operação. Cada domínio define o seu próprio conjunto de erros, o que torna o tratamento de falhas explícito e evita o recurso a exceções não controladas. As operações de listagem seguem ainda um padrão uniforme de paginação, através das abstrações `Page` e `PageRequest`.

Existem vários serviços, cada um associado a uma área funcional da aplicação:

- **UserService:** gere os utilizadores e as respetivas sessões, incluindo a criação de utilizadores, a autenticação e a gestão de *tokens*;
- **MemberService:** gere o ciclo de vida dos sócios, incluindo a criação, aprovação, rejeição, ativação e desativação, bem como a alteração de dados e de categoria;
- **AthleteService:** gere a inscrição e o ciclo de vida dos atletas. A inscrição de um atleta cria, numa única transação, o sócio associado, o atleta e os respetivos encarregados de educação;
- **EventService:** gere os jogos e a respetiva bilhética, incluindo a criação de eventos e setores e o início do processo de compra de bilhetes;
- **PaymentService:** coordena os pagamentos, as quotas e a emissão de recibos, e integra-se com o serviço de pagamentos externo;
- **SponsorService** e **SponsorshipService:** gerem, respetivamente, os patrocinadores e os contratos de patrocínio;
- **SponsorshipCatalogService** e **TeamService:** gerem os catálogos de opções de patrocínio e a organização das equipas e tabelas de preços;
- **AdminOverviewService:** agrega estatísticas para o painel de administração;
- **FileService:** gere o carregamento, a consulta e a remoção de ficheiros, aplicando validações de tipo e tamanho e verificando as permissões de acesso;

- **EmailService**: compõe e envia mensagens de correio eletrónico.

A maioria destes serviços segue um modelo simples e consistente: recebem os dados, validam as regras de negócio, interagem com os repositórios correspondentes e retornam o resultado da operação. Por esse motivo, não são detalhados individualmente. O **PaymentService**, pela sua complexidade e pela integração com serviços externos, é descrito em detalhe na Secção 4.6.

4.4 Controladores

No contexto da *framework* Spring, um controlador (ou *controller*) é o componente responsável por receber e tratar os pedidos HTTP efetuados pelos clientes. Estes pedidos são originados a partir da aplicação *frontend*, que comunica com o *backend* através de uma Web API REST. A função principal de um controlador é interpretar os dados recebidos, invocar os serviços adequados e devolver uma resposta apropriada, normalmente em formato JSON. Para este efeito, foi utilizada a anotação **@RestController**, apropriada para aplicações que não retornam páginas HTML, mas sim dados em formato JSON.

Existem vários controladores na aplicação, cada um responsável por um conjunto específico de *endpoints*. Os principais são:

- **UserController**: criação de utilizadores, autenticação (*login* e *logout*) e obtenção da informação do utilizador autenticado;
- **MemberController**: criação, consulta, atualização e gestão do ciclo de vida dos sócios;
- **AthleteController**: inscrição de atletas, consulta pública por escalão e gestão administrativa dos atletas;
- **EventController**: criação e gestão de jogos, consulta de eventos disponíveis e início do processo de compra de bilhetes;
- **PaymentController**: criação de sessões de pagamento, gestão de quotas, emissão de recibos e receção de notificações do serviço de pagamentos;
- **SponsorController** e **SponsorshipController**: gestão de patrocinadores e de contratos de patrocínio;
- **SponsorshipCatalogController** e **TeamController**: gestão dos catálogos de patrocínio e da organização das equipas;
- **AdminController**: obtenção de estatísticas para o painel de administração;
- **FileController**: carregamento, consulta e remoção de ficheiros.

Importa notar que os controladores que requerem autenticação recebem um objeto com a informação do utilizador autenticado, injetado automaticamente pela aplicação, como detalhado na Secção 4.5.

Além da separação clara de responsabilidades, os controladores seguem um estilo coeso de tratamento de erros. A aplicação adota o padrão *Problem Details for HTTP APIs* (RFC 7807), que devolve respostas de erro com uma estrutura unificada em formato JSON, com campos como `type`, `title`, `status` e `detail`, que permitem compreender de forma padronizada a natureza do erro ocorrido. A conversão dos erros internos de cada domínio para este formato é feita de forma centralizada.

4.5 Autenticação e Autorização

A autenticação dos utilizadores assenta num mecanismo de *tokens* opacos, gerados pela aplicação e persistidos na base de dados. Ao contrário de uma palavra-passe, o *token* não transporta qualquer informação legível: é apenas uma credencial aleatória que o servidor sabe associar a um utilizador. As palavras-passe dos utilizadores nunca são armazenadas em texto simples, sendo guardada apenas a sua representação cifrada através do algoritmo *BCrypt*.

O processo de autenticação segue os seguintes passos:

1. O utilizador submete as suas credenciais (nome de utilizador ou *email*, e palavra-passe);
2. O servidor procura o utilizador correspondente e valida a palavra-passe submetida contra a representação cifrada armazenada;
3. Em caso de sucesso, é gerado um *token* aleatório, do qual é guardada na base de dados apenas uma representação cifrada (*hash* SHA-256);
4. O *token* é devolvido ao cliente, tanto no corpo da resposta como num *cookie* seguro (`HttpOnly`), utilizado nos pedidos seguintes.

Cada *token* tem associado um tempo de validade absoluto e um tempo de validade por inatividade, sendo ainda limitado o número máximo de *tokens* ativos por utilizador. A validação dos pedidos autenticados é feita de forma centralizada por um *interceptor*, que é executado antes dos controladores. Este componente extrai o *token* do cabeçalho de autorização (ou, em alternativa, do *cookie*), valida-o e, em caso de sucesso, disponibiliza a informação do utilizador autenticado ao controlador. Caso a validação falhe, o pedido é rejeitado com o código de estado HTTP 401.

No que diz respeito à autorização, a aplicação define três papéis: **ADMIN**, **SECRETARIA** e **NORMAL**. Os papéis **ADMIN** e **SECRETARIA** partilham o acesso às funcionalidades de gestão (*backoffice*), enquanto o papel **NORMAL** corresponde ao utilizador comum. A verificação de permissões é feita ao nível dos serviços e dos controladores. Para além da verificação por

papel, existe também uma verificação de propriedade do recurso, que garante, por exemplo, que um utilizador só pode pagar ou consultar dados que lhe dizem respeito.

4.6 Integrações Externas

O *backend* integra-se com vários serviços externos para suportar funcionalidades que não fazem parte do seu domínio central. Esta secção descreve essas integrações.

4.6.1 Pagamentos com Stripe

O processamento de pagamentos é feito através da plataforma Stripe [6], que trata de toda a interação com os meios de pagamento, sem que os dados bancários dos utilizadores passem pelo servidor da aplicação. A integração é coordenada pelo `PaymentService` e suporta diferentes tipos de cobrança: quotas de sócio, mensalidades de atleta, patrocínios e compra de bilhetes.

O fluxo de pagamento decorre da seguinte forma:

1. O cliente inicia o pagamento através do *endpoint* correspondente;
2. O *backend* cria a cobrança e os respetivos itens, regista um pagamento no estado pendente e cria uma sessão de pagamento na Stripe, devolvendo ao cliente o endereço para onde deve ser redirecionado;
3. Após a conclusão do pagamento, a Stripe notifica o *backend* através de um *webhook*;
4. O *backend* valida a autenticidade da notificação e, em caso de sucesso, confirma o pagamento e a cobrança, atualiza o estado das entidades associadas (por exemplo, confirma os bilhetes e gera os respetivos códigos QR) e envia uma mensagem de confirmação ao utilizador. Em caso de falha, o pagamento é marcado como falhado e os recursos reservados são libertados.

A validação da autenticidade das notificações é feita através da verificação de uma assinatura, o que impede que pedidos forjados sejam aceites como pagamentos válidos.

4.6.2 Envio de Correio Eletrónico

O envio de mensagens de correio eletrónico é suportado pelo módulo de *email* do Spring, configurado para comunicar com um servidor SMTP externo. A composição das mensagens é da responsabilidade do `EmailService`, que gera conteúdo em HTML. A principal utilização atual é o envio da confirmação de compra de bilhetes, na qual os códigos QR são incluídos como imagens incorporadas na própria mensagem.

4.6.3 Armazenamento de Ficheiros

O conteúdo binário dos ficheiros não é guardado na base de dados, mas sim num serviço de armazenamento externo. A aplicação utiliza o serviço Cloudflare R2, acedido através de uma API compatível com o protocolo S3 da Amazon. O acesso ao armazenamento é abstraído por uma interface, o que permite, no futuro, substituir o serviço utilizado sem alterar a lógica da aplicação. Este serviço é utilizado para armazenar fotografias de perfil, fotografias de sócios e atletas, e documentos associados aos atletas.

4.6.4 Geração de Códigos QR

A geração dos códigos QR utilizados na bilhética é feita através da biblioteca ZXing [7]. A partir de um identificador único associado a cada bilhete, é gerada uma imagem que é posteriormente incluída na mensagem de confirmação de compra enviada ao utilizador.

Capítulo 5

Frontend

O *frontend* da aplicação tem como principal objetivo fornecer uma interface intuitiva e interativa para que os utilizadores possam aceder às funcionalidades do clube, nomeadamente a adesão de sócios, a inscrição de atletas, o registo de patrocínios e a compra de bilhetes, bem como permitir a gestão destas áreas por parte dos utilizadores responsáveis.

Esta interface é construída como uma SPA (*Single Page Application*), ou seja, uma aplicação de página única, na qual a navegação entre as diferentes secções não implica o recarregamento da página. Foi desenvolvida na linguagem TypeScript, uma linguagem fortemente tipificada que se baseia em JavaScript e que permite uma maior segurança em tempo de desenvolvimento e a deteção antecipada de erros. Para a construção da interface foi utilizada a biblioteca React [8], que permite a construção de interfaces baseadas em componentes reutilizáveis, com atualização e renderização eficientes através de uma DOM virtual.

O empacotamento e o ambiente de desenvolvimento são geridos pela ferramenta Vite [9], que oferece um servidor de desenvolvimento rápido e um processo de *build* otimizado. Durante o desenvolvimento, o Vite está ainda configurado para reencaminhar os pedidos dirigidos à API para o servidor do *backend*, o que simplifica a comunicação entre as duas componentes.

No que diz respeito ao aspeto visual, a aplicação combina duas abordagens. Por um lado, recorre ao Tailwind CSS [10], uma *framework* de CSS utilitária que define um tema próprio (paleta de cores, tipografia e animações) alinhado com a identidade visual do clube. Por outro, utiliza folhas de estilo de CSS próprias, organizadas por funcionalidade, para os estilos mais específicos de cada área da aplicação.

5.1 Estrutura da Aplicação

A aplicação está organizada de forma modular, separada por domínio funcional. O código divide-se em duas grandes áreas: uma pasta de código transversal, partilhado por toda a aplicação, e uma pasta de funcionalidades, na qual cada domínio (atletas, sócios, patrocínios, eventos, utilizadores, entre outros) constitui uma unidade autocontida.

Cada funcionalidade agrupa os seus próprios componentes, tipos de dados, estilos, lógica

de orquestração e o acesso à API. Esta organização mantém próximos todos os elementos relativos a uma mesma área, o que facilita a manutenção e a evolução do código. Entre os elementos transversais, partilhados por todas as funcionalidades, incluem-se componentes reutilizáveis (como o cabeçalho, o rodapé e os campos de formulário), funções utilitárias, a configuração da aplicação e a gestão da autenticação.

A comunicação com o *backend* é feita através de uma camada de acesso à Web API, separada por entidade. Cada funcionalidade define as operações de que necessita sobre uma função auxiliar comum, que efetua os pedidos HTTP, inclui as credenciais de sessão e converte as respostas de erro do servidor numa exceção uniforme. Desta forma, os componentes consomem funções bem definidas em vez de lidarem diretamente com os detalhes da comunicação em rede.

A lógica de cada área é frequentemente encapsulada em *hooks* personalizados, que combinam o carregamento de dados a partir do *backend* com a gestão do estado da interface. Esta abordagem baseada em *hooks* permite uma estrutura declarativa, na qual a interface reflete o estado da aplicação a cada momento.

5.2 Navegação e Autenticação

A navegação entre as páginas da aplicação é gerida pela biblioteca React Router [11], que define as rotas da aplicação num único ponto e permite a composição de *layouts* reutilizáveis partilhados por várias páginas.

O estado de autenticação do utilizador é mantido de forma global através do mecanismo de contexto do React. Ao iniciar, a aplicação consulta o *backend* para determinar se existe uma sessão ativa e, em caso afirmativo, carrega a informação do utilizador autenticado, que fica acessível a toda a aplicação. A sessão é suportada por *cookies* geridos pelo *backend*, enviados automaticamente em todos os pedidos.

A aplicação adapta-se ao papel do utilizador autenticado (ADMIN, SECRETARIA ou NORMAL). Esta adaptação ocorre a dois níveis. Ao nível da navegação, as rotas são protegidas por componentes que verificam, antes de apresentar uma página, se o utilizador está autenticado e se possui o papel necessário, redirecionando-o caso contrário. Toda a área de gestão (*backoffice*) está protegida desta forma, acessível apenas a utilizadores com os papéis ADMIN ou SECRETARIA. Ao nível da interface, vários elementos são apresentados condicionalmente consoante o papel do utilizador, como por exemplo as ligações para a área de gestão ou os campos adicionais disponíveis apenas aos responsáveis do clube.

5.3 Interface da Aplicação

A aplicação apresenta uma navegação simples entre as principais áreas, através de um menu superior. Podem distinguir-se duas grandes vertentes: uma área pública, orientada para o

visitante e para o sócio, e uma área de gestão, orientada para os utilizadores responsáveis pela administração do clube.

Na área pública encontram-se a página inicial, os fluxos de adesão de sócio, inscrição de atleta e registo de patrocínio, a consulta de atletas por escalão e a compra de bilhetes para os jogos. Na área de gestão é possível consultar e gerir sócios, atletas, patrocínios e eventos, administrar os utilizadores da aplicação e configurar os catálogos de opções e as tabelas de preços. Esta área dispõe ainda de um painel inicial com estatísticas gerais sobre o estado do clube.

5.4 Comunicação com a API

A comunicação entre o *frontend* e o *backend* é realizada através de pedidos HTTP para a Web API REST disponibilizada pelo servidor. Para centralizar esta comunicação, cada domínio funcional possui o seu próprio ficheiro de acesso à API, como por exemplo `auth/api.ts`, `Members/api.ts`, `Athletes/api.ts`, `sponsors/api.ts`, `events/api.ts`, `files/api.ts` e `admin/api.ts`.

Apesar de cada funcionalidade definir as operações específicas de que necessita, a estrutura dos pedidos segue um padrão comum. O endereço base da API é definido num ficheiro de configuração partilhado, através da constante `BASE_URL`, cujo valor é `/api`. Desta forma, os componentes da aplicação não dependem diretamente do endereço absoluto do servidor, o que facilita a execução em diferentes ambientes, como desenvolvimento local ou Docker.

Durante o desenvolvimento, o servidor Vite reencaminha automaticamente os pedidos feitos para `/api` para o servidor do *backend*. Este mecanismo de *proxy* evita problemas de CORS (*Cross-Origin Resource Sharing*) e permite que o *frontend* comunique com o *backend* como se ambos estivessem disponíveis no mesmo domínio.

Todos os pedidos autenticados incluem a opção `credentials`: `"include"`, permitindo que os *cookies* de sessão definidos pelo *backend* sejam enviados automaticamente. Esta decisão mantém a lógica de autenticação centrada no servidor, reduzindo a necessidade de manipulação manual de tokens no *frontend*.

O tratamento de erros também segue uma abordagem uniforme. Quando uma resposta da API indica erro, a camada de acesso converte essa resposta numa exceção ou num objeto de erro coerente, que é posteriormente tratado pelos componentes ou pelos *hooks*. Esta abordagem evita que cada componente tenha de interpretar manualmente códigos HTTP e formatos de erro diferentes.

5.5 Gestão de Estado e Hooks

A aplicação utiliza o estado local do React para gerir a informação necessária em cada componente, como valores de formulários, mensagens de erro, estados de carregamento e resultados

de operações. Para evitar duplicação de lógica, foram criados vários *hooks* personalizados, organizados por funcionalidade.

Estes *hooks* encapsulam operações como o carregamento de listas, a consulta de detalhes, a criação e atualização de entidades e a execução de ações administrativas. Por exemplo, existem *hooks* para carregar sócios, consultar detalhes de um atleta, gerir catálogos de patrocínio, obter estatísticas de administração ou consultar eventos.

Esta organização traz várias vantagens:

- separa a lógica de carregamento de dados da lógica visual dos componentes;
- reduz a repetição de código entre páginas semelhantes;
- torna os componentes mais declarativos e fáceis de compreender;
- facilita a manutenção, uma vez que alterações na forma de comunicar com a API ficam concentradas nos *hooks* ou nos ficheiros de API.

Além dos *hooks* específicos de cada domínio, existem também *hooks* partilhados, como `useAuth`, utilizado para aceder ao estado global de autenticação, e `useStatusHandler`, utilizado para tratar estados comuns de operações assíncronas, como carregamento, sucesso e erro.

5.6 Componentes Reutilizáveis

A aplicação recorre a vários componentes reutilizáveis, localizados na pasta `shared/components`. Estes componentes representam elementos comuns da interface, utilizados em várias áreas da aplicação.

Entre os principais componentes partilhados destacam-se:

- **Header**: cabeçalho da aplicação, responsável pela navegação principal e pela apresentação de opções dependentes do estado de autenticação;
- **Footer**: rodapé da aplicação, contendo informação institucional e ligações relevantes;
- **Require** e **AuthRequire**: componentes responsáveis por proteger rotas de acordo com o estado de autenticação e o papel do utilizador;
- **Form**: componente auxiliar para estruturar formulários;
- **LabeledField**: componente utilizado para apresentar campos com rótulo de forma consistente;
- **MessageFormBox**: componente utilizado para apresentar mensagens de erro, sucesso ou informação ao utilizador.

A existência destes componentes contribui para uma interface mais consistente e reduz a repetição de código. Por exemplo, os formulários de criação e edição de sócios ou atletas partilham padrões visuais e comportamentais semelhantes, permitindo ao utilizador reconhecer rapidamente a forma de interação esperada.

5.7 Rotas da Aplicação

A navegação da aplicação é definida de forma centralizada no ficheiro `jagozRouter.tsx`, através da função `createBrowserRouter` da biblioteca `React Router`. Esta configuração descreve todas as páginas disponíveis e a forma como estão organizadas.

As principais rotas públicas incluem:

- `/`: página inicial da aplicação;
- `/auth/login`: página de autenticação;
- `/auth/register`: página de registo de utilizador;
- `/athletes/category/:teamCategory`: listagem pública de atletas por escalão;
- `/athletes/:athleteId`: perfil público de um atleta;
- `/sponsors/info`: página informativa sobre patrocínios;
- `/tickets`: listagem pública de eventos disponíveis para compra de bilhetes;
- `/tickets/:eventId`: fluxo de compra de bilhetes para um evento.

Existem também rotas protegidas, acessíveis apenas a utilizadores autenticados, como:

- `/profile`: perfil do utilizador autenticado;
- `/members/create`: adesão ou criação de sócio;
- `/athletes/register`: inscrição de atleta;
- `/sponsors/create`: criação de pedido de patrocínio;
- `/sponsors/my`: consulta dos patrocínios associados ao utilizador.

Por fim, a área de administração encontra-se agrupada sob a rota `/admin`. Esta área é protegida por componentes de autorização e apenas pode ser acedida por utilizadores com os papéis `ADMIN` ou `SECRETARIA`.

- `/admin`: Painel inicial de administração;
- `/admin/members`: Gestão de sócios;

- `/admin/athletes`: Gestão de atletas;
- `/admin/users`: Gestão de utilizadores;
- `/admin/sponsors`: Gestão de patrocínios;
- `/admin/events`: Gestão de eventos;
- `/admin/team-settings`: Configuração de equipas e escalões;

Para além destas rotas principais, existem rotas auxiliares para operações específicas, como criação, edição e consulta de detalhe. Por exemplo, a gestão de sócios inclui rotas para criar um novo sócio, consultar o detalhe de um sócio existente e editar os seus dados. Esta organização evita concentrar todas as operações numa única página e permite separar claramente cada fluxo de utilização.

5.8 Funcionalidades por Domínio

O *frontend* encontra-se organizado por funcionalidades, cada uma representando uma área do sistema. Esta organização permite que cada domínio tenha os seus próprios componentes, tipos, estilos, *hooks* e chamadas à API.

5.8.1 Página Inicial

A página inicial funciona como ponto de entrada da aplicação. Apresenta a identidade visual do clube, uma área de destaque e secções informativas, como notícias e classificações. Esta página tem uma função essencialmente pública, servindo para orientar visitantes e utilizadores para as principais ações disponíveis, como inscrição, consulta de atletas, patrocínios e compra de bilhetes.

A componente visual desta página utiliza imagens públicas do clube e estilos próprios, definidos na funcionalidade `home`. Esta área inclui componentes como `Hero`, `NewsGrid` e `StandingTable`.



Figura 5.1: Página inicial da aplicação

5.8.2 Autenticação

A funcionalidade de autenticação inclui as páginas de início de sessão e de registo. O início de sessão permite ao utilizador autenticar-se com as suas credenciais, recebendo do *backend* uma sessão válida. Após autenticação, a informação do utilizador é guardada no contexto global da aplicação.

O registo permite criar uma nova conta de utilizador. Esta conta pode depois ser associada a um sócio, atleta ou patrocinador, dependendo do fluxo de utilização. Esta separação entre conta de utilizador e entidades do clube permite maior flexibilidade, uma vez que nem todos os utilizadores são necessariamente sócios ou patrocinadores no momento da criação da conta.

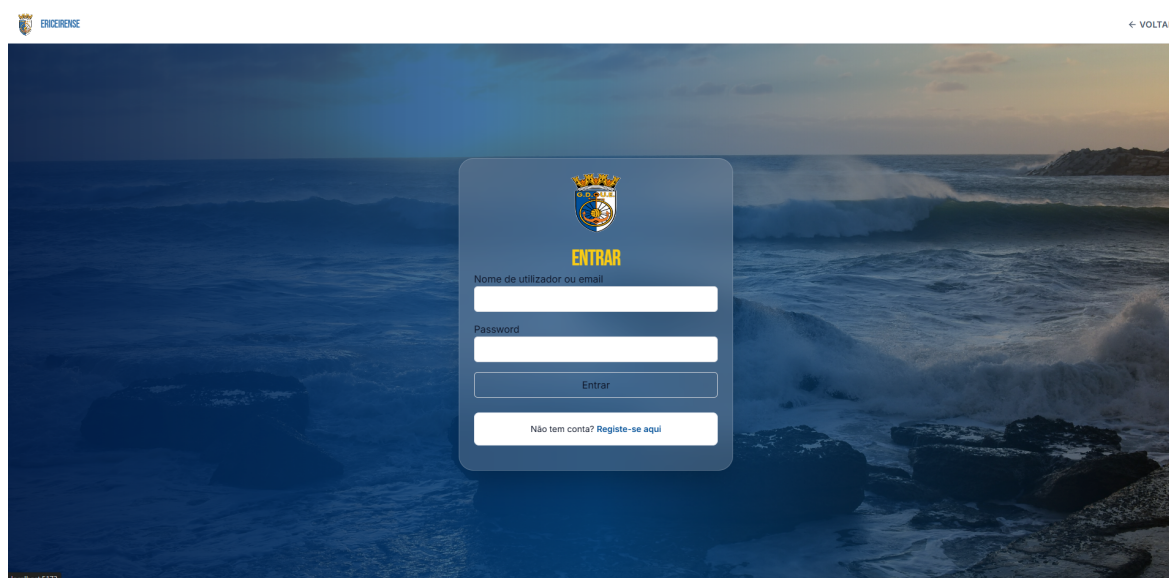


Figura 5.2: Página de autenticação

5.8.3 Sócios

A área de sócios permite criar, consultar, editar e gerir sócios do clube. No lado público ou autenticado, o utilizador pode preencher o formulário de adesão, introduzindo dados pessoais, contactos, morada, NIF, categoria e consentimentos.

Na área administrativa, os utilizadores com permissões podem consultar a lista de sócios, aplicar filtros, aceder ao detalhe de cada sócio e executar ações de gestão, como aprovação, rejeição, reativação, alteração de categoria ou edição dos dados.

Esta funcionalidade inclui também a apresentação de informação financeira associada ao sócio, nomeadamente quotas ou pagamentos pendentes, quando aplicável. O acesso a recibos é feito através de ligações para os *endpoints* de pagamento disponibilizados pelo *backend*.

criar novo sócio

Formulário de candidatura com os dados necessários para avaliação por parte do clube. [← VOLTAR](#)

FOTO DE SÓCIO
Esta foto será usada como imagem principal do perfil.
[ESCOLHER FOTO](#)

CONTA ASSOCIADA
Username da conta
Opcional. Se preencheres, o backend procura esse username e associa a conta ao sócio.

NOME COMPLETO

DATA DE NASCIMENTO

EMAIL

TELEMÓVEL

TELEFONE DE CASA

NIF

CATEGORIA

QUOTA MENSAL (EUR)

Valor mínimo para sócio: 1,50 EUR

MORADA

CÓDIGO POSTAL

CIDADE

LOCAL DE COBRANÇA

Ex: Sede do clube

☐ Já foi sócio anteriormente
Ajuda a secretaria a perceber se a inscrição é nova ou uma reentrada.

Texto de privacidade obrigatório
Os dados pessoais recolhidos neste formulário serão tratados pelo Grupo Desportivo União Enxaimense para efeitos de inscrição como sócio do clube e para envio de comunicações relacionadas com a atividade desportiva. Poderá exercer a qualquer momento, os direitos de acesso, retificação, avaliação, oposição ou eliminação dos seus dados pessoais, nos casos

[LI O TEXTO](#)

Figura 5.3: Formulário de adesão de sócio

5.8.4 Atletas

A funcionalidade de atletas suporta dois tipos principais de utilização. Por um lado, permite a inscrição de novos atletas através de um formulário detalhado. Este formulário recolhe dados pessoais, informação desportiva, dados escolares, informação médica/administrativa e dados dos encarregados de educação.

Por outro lado, permite a consulta pública de atletas por escalão. Esta consulta expõe apenas informação não sensível, como nome, número, posição, nacionalidade, idade e escalão. Dados sensíveis, como documentos, contactos, NIF, número de identificação ou dados de encarregados de educação, ficam reservados à área administrativa.

Na área de gestão, os responsáveis podem consultar todos os atletas, incluindo pendentes e rejeitados, editar dados administrativos, alterar o escalão, aprovar ou rejeitar inscrições e ativar ou desativar atletas.

Existe ainda uma página de configuração de equipas, onde é possível gerir grupos, escalões e tabelas de preços utilizadas nas opções de patrocínio associadas a equipas.

The image shows a web interface for an athlete's profile. At the top, there is a header section with a circular profile picture of a man, the name "DIOGO PEREIRA", and the text "Sócio #1007 - Atleta #5". Below this are two buttons: "ALTERAR FOTO" and "ATUALIZAR". To the right of the name are two status buttons: "ATIVO" (green) and "VETERANOS" (blue). Below the header is a section titled "IDENTIDADE" containing a table of personal information:

DATA DE NASCIMENTO	NATURALIDADE
25/06/1985	—
NACIONALIDADE	EMAIL
Portuguesa	diogo.pereira@example.pt
TELEMÓVEL	NÚMERO
912345007	Não atribuído
POSIÇÃO	
Não atribuída	

Below the identity section is a "DOCUMENTOS" section with a table of documents:

NIF	NISS
111111117	11122233305
Nº UTENTE	BI / CC / PASSAPORTE
300003005	CC30005
VALIDADE BI	
05/11/2028	

At the bottom of the documents section, there are two upload areas:

- CARTÃO DE CIDADÃO**: A button labeled "CARREGAR" and a file name "pp008.pdf.pdf".
- EXAME MEDICO**: A button labeled "CARREGAR" and the text "Ainda não foi carregado nenhum ficheiro."

Figura 5.4: Página de perfil de atleta

5.8.5 Patrocínios

A área de patrocínios permite que um utilizador consulte informação sobre as opções disponíveis e submeta um pedido de patrocínio. O sistema suporta diferentes tipos de patrocínio, como publicidade em lona/outdoor, patrocínio de equipas e apoio a outras modalidades ou iniciativas.

O *frontend* apresenta os catálogos de opções disponíveis, calcula os valores associados e permite a submissão do pedido. Para utilizadores autenticados, existe também uma área de consulta dos seus próprios patrocínios, onde podem acompanhar o estado de cada pedido e aceder aos detalhes ou recibos.

Na área administrativa, os responsáveis podem consultar todos os patrocínios, aprovar pedidos, marcar pagamentos, cancelar contratos, gerir patrocinadores e configurar os catálogos de opções. Esta configuração inclui opções de publicidade, colocações em equipamentos, outras modalidades e regras de preços por grupo ou escalão.

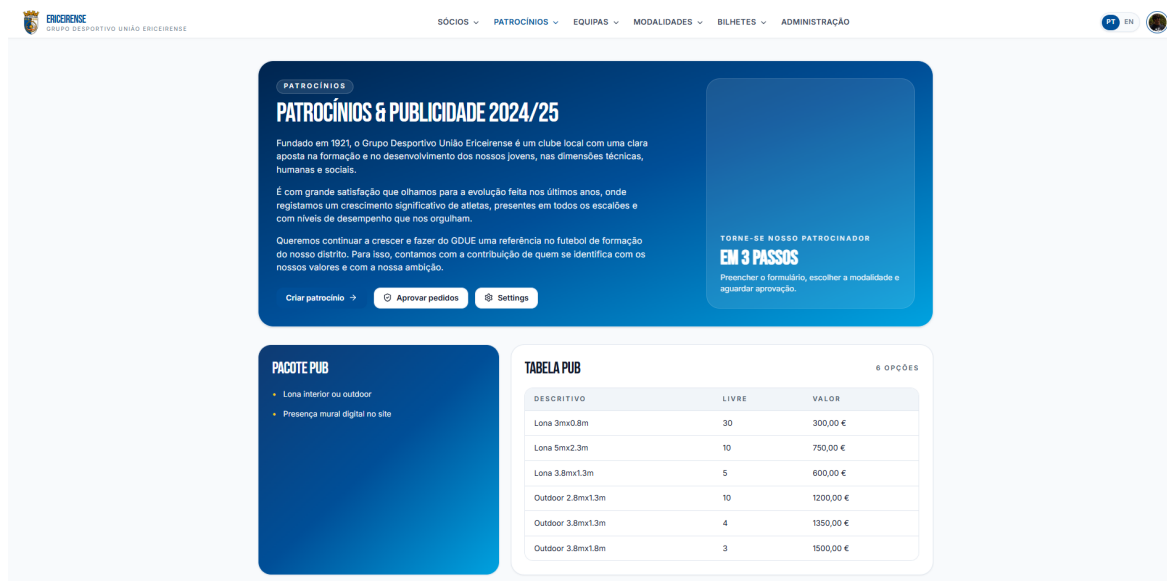


Figura 5.5: Página de Informação sobre Patrocínios

5.8.6 Eventos e Bilheteira

A funcionalidade de eventos permite criar e gerir jogos ou outros eventos do clube. Na área administrativa, é possível criar eventos, definir descrição, local, data, preços para público geral e sócios, e configurar setores com lotação própria.

A área pública de bilheteira permite consultar eventos disponíveis e iniciar o processo de compra de bilhetes. Durante este processo, o utilizador seleciona os setores pretendidos e informa os dados necessários para a compra. Quando o bilhete é de sócio, o sistema suporta a validação do número de sócio e da data de nascimento, garantindo que o desconto é aplicado apenas a sócios válidos.

Após a criação da sessão de pagamento, o utilizador é redirecionado para o serviço de pagamento. O *frontend* possui ainda páginas específicas para o resultado do pagamento, nomeadamente sucesso e cancelamento, acessíveis pelas rotas `/payments/success` e `/payments/cancel`.

5.8.7 Utilizadores e Perfil

A área de utilizadores permite ao utilizador autenticado consultar o seu perfil e as entidades a si associadas, como sócio, atleta ou patrocinadores. Na área administrativa, os responsáveis podem consultar a lista de utilizadores, aceder ao detalhe de cada um, alterar o papel atribuído e associar um sócio ativo.

Esta funcionalidade é importante para ligar a autenticação às entidades do domínio do clube. Um utilizador pode ter permissões diferentes consoante o seu papel, mas também pode estar associado a um sócio ou patrocinador, o que influencia as ações que pode realizar.

5.8.8 Ficheiros

A aplicação inclui uma funcionalidade dedicada à gestão de ficheiros, utilizada por várias áreas do sistema. Esta funcionalidade permite listar, carregar, consultar e remover ficheiros associados a entidades como utilizadores, sócios ou atletas.

No *frontend*, esta gestão é encapsulada em componentes e funções de API próprias. Os ficheiros são enviados através de pedidos `multipart/form-data`, adequados ao envio de conteúdo binário. Para visualização ou acesso aos ficheiros, a aplicação constrói ligações para os *endpoints* do *backend*, como o conteúdo de um ficheiro ou a fotografia pública de um atleta.

A utilização desta funcionalidade permite manter a gestão documental integrada na aplicação, sem misturar o conteúdo binário com os dados estruturados apresentados nos formulários.

5.8.9 Administração

A área de administração funciona como o *backoffice* da aplicação. O seu acesso está reservado a utilizadores com os papéis ADMIN ou SECRETARIA. Esta área reúne as ferramentas necessárias para gerir o funcionamento do clube.

O painel inicial apresenta estatísticas gerais, permitindo obter uma visão rápida sobre o estado da organização, como número de sócios, atletas, patrocínios ou pedidos pendentes. A partir deste painel, os responsáveis podem navegar para as diferentes áreas de gestão: sócios, atletas, utilizadores, patrocínios, eventos e configurações.

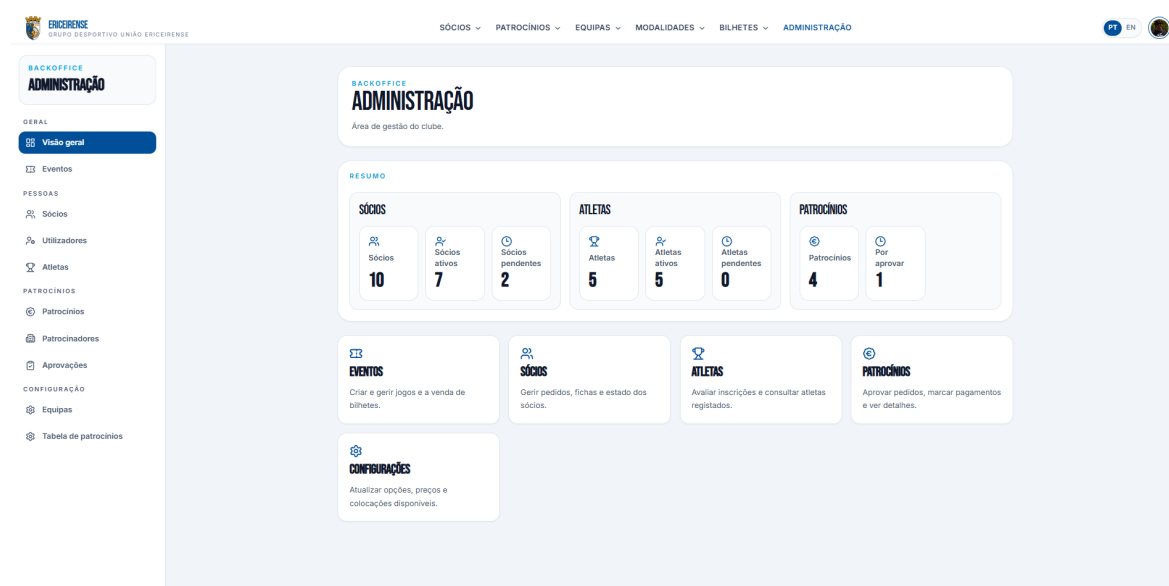


Figura 5.6: Página Inicial do BackOffice

5.9 Formulários e Validação

Uma parte significativa da aplicação é composta por formulários, uma vez que os principais processos do clube envolvem recolha de dados: adesão de sócios, inscrição de atletas, criação de patrocínios, edição de utilizadores e criação de eventos.

Os formulários são estruturados em componentes específicos, como `MemberForm` e `AthleteForm`, e utilizam tipos TypeScript para representar os dados esperados. Para facilitar a inicialização e transformação dos valores dos formulários, existem ficheiros utilitários como `formValues.ts`, que concentram valores por omissão e conversões necessárias antes do envio para a API.

A validação é feita em dois níveis. No *frontend*, são aplicadas validações imediatas que melhoram a experiência do utilizador, como campos obrigatórios, formatos de datas, valores numéricos e conversões monetárias. No *backend*, as regras de negócio são novamente validadas, garantindo que a integridade dos dados não depende apenas da interface.

Esta dupla validação é importante porque o *frontend* melhora a usabilidade, mas o *backend* continua a ser a fonte de verdade das regras da aplicação.

5.10 Tipos e Segurança em Tempo de Desenvolvimento

A utilização de TypeScript permite declarar tipos para os dados manipulados pela aplicação. Cada funcionalidade define os seus próprios tipos, como `Member`, `Athlete`, `Sponsorship`, `Event` ou tipos auxiliares de formulários.

Esta tipagem contribui para reduzir erros durante o desenvolvimento. Por exemplo, se uma função espera um identificador numérico ou um determinado campo obrigatório, o compilador consegue assinalar usos incorretos antes da execução da aplicação. Isto torna o código mais previsível e facilita a refatoração.

A tipagem é especialmente útil na comunicação com a API, uma vez que as respostas do *backend* são convertidas para estruturas conhecidas no *frontend*. Embora esta validação não substitua a validação no servidor, ajuda a manter consistência entre as duas camadas.

5.11 Tratamento de Estados da Interface

As operações assíncronas, como carregamento de listas, submissão de formulários ou atualização de dados, exigem a representação de diferentes estados na interface. A aplicação distingue estados como carregamento, sucesso, erro e ausência de dados.

Esta distinção permite fornecer uma experiência mais clara ao utilizador. Por exemplo, durante o carregamento de uma lista, a interface pode apresentar uma mensagem ou indicador de espera; em caso de erro, pode mostrar uma mensagem descritiva; e após uma operação bem-sucedida, pode redirecionar o utilizador ou apresentar confirmação.

A existência de funções e *hooks* auxiliares para este tratamento evita que cada componente implemente de forma diferente a mesma lógica de estado.

5.12 Estilo Visual e Responsividade

O estilo visual da aplicação combina CSS próprio com Tailwind CSS. As folhas de estilo específicas de cada funcionalidade permitem controlar detalhes visuais de páginas complexas, como formulários, painéis de administração, cartões, listas e tabelas. O Tailwind CSS é utilizado como base utilitária e para definir um tema coerente.

A aplicação procura refletir a identidade visual do clube, recorrendo a imagens e elementos gráficos presentes na pasta pública, como o logótipo e imagens de fundo. Estes elementos ajudam a aproximar a interface da identidade institucional do GDUE.

A responsividade é uma preocupação importante, uma vez que a aplicação pode ser utilizada tanto em computadores como em dispositivos móveis. A interface é organizada para se adaptar a diferentes larguras de ecrã, especialmente nas áreas públicas, como a página inicial, consulta de atletas, inscrição e compra de bilhetes. Nas áreas administrativas, onde existe maior densidade de informação, a prioridade é manter a leitura clara e a navegação eficiente.

5.13 Internacionalização

Uma das funcionalidades relevantes do *frontend* é o suporte a múltiplos idiomas, através de um módulo de internacionalização (i18n). Esta funcionalidade foi considerada importante no contexto do clube, uma vez que a Ericeira conta atualmente com uma comunidade internacional significativa: o clube integra cerca de 20 nacionalidades diferentes, na sua maioria sem o português como língua oficial. Neste contexto, o suporte ao idioma inglês tornou-se um requisito mínimo para garantir o acesso desta comunidade às funcionalidades da aplicação.

A internacionalização foi implementada com recurso à biblioteca i18next [12] e à sua integração com o React. A aplicação suporta dois idiomas, português e inglês, mantidos em paridade. As traduções de cada idioma estão organizadas num ficheiro próprio, estruturado em secções correspondentes às diferentes áreas da aplicação.

A seleção do idioma é automática na primeira utilização: a aplicação deteta o idioma preferido do utilizador e, na ausência de informação, assume o português por omissão. A escolha do utilizador é memorizada para utilizações seguintes. É ainda possível alternar manualmente entre os idiomas disponíveis através de um seletor presente no cabeçalho da aplicação. Os componentes consomem as traduções através de uma função dedicada, que permite também a inserção de valores dinâmicos no texto traduzido.

Capítulo 6

Conclusão

O desenvolvimento da JagozApp permitiu construir uma solução web orientada à gestão de processos administrativos e operacionais de um clube desportivo local. Ao longo do projeto foi possível centralizar num único sistema funcionalidades que, tradicionalmente, tendem a estar dispersas por diferentes ferramentas ou processos manuais, nomeadamente a gestão de sócios, atletas, equipas, patrocinadores, eventos, bilheteira, pagamentos e documentos.

A aplicação resultante é composta por um *backend* desenvolvido em Kotlin com Spring Boot e por um *frontend* desenvolvido em React com TypeScript e Vite. Esta separação entre cliente e servidor permitiu definir responsabilidades claras: o *backend* concentra a lógica de negócio, persistência, autenticação, autorização e integrações externas, enquanto o *frontend* disponibiliza uma interface web para os diferentes perfis de utilização. A utilização de PostgreSQL como base de dados relacional, JDBI para acesso a dados, Stripe para pagamentos, Cloudflare R2 para armazenamento de ficheiros e Docker para execução local contribuiu para uma solução modular, extensível e próxima de um ambiente real de produção.

Um dos aspetos mais relevantes do projeto foi a organização modular do *backend*. A separação entre os módulos de domínio, serviços, repositórios, camada HTTP e arranque da aplicação permitiu reduzir o acoplamento entre componentes e facilitar a testabilidade. Esta decisão revelou-se especialmente importante devido à dimensão funcional da aplicação, que envolve várias áreas de negócio relacionadas entre si. A utilização de interfaces para os repositórios e de um **TransactionManager** permitiu ainda isolar a lógica de negócio da tecnologia concreta de persistência.

No *frontend*, a organização por domínio funcional permitiu manter próximos os componentes, tipos, estilos, *hooks* e chamadas à API associados a cada área da aplicação. Esta abordagem facilitou a evolução da interface e permitiu criar fluxos distintos para utilizadores públicos, utilizadores autenticados e utilizadores com permissões administrativas. A existência de rotas protegidas, contexto de autenticação e internacionalização contribuiu para uma experiência mais completa e adequada ao contexto do clube.

Relativamente aos objetivos definidos, considera-se que os requisitos principais foram cumpridos. A aplicação permite a criação e gestão de sócios, inscrição e aprovação de atletas,

gestão de patrocinadores e contratos de patrocínio, criação de eventos, compra de bilhetes, integração com pagamentos online, gestão de ficheiros e administração das principais entidades do sistema. Para além disso, foram produzidos elementos de apoio importantes, como documentação da API em OpenAPI, ficheiros de configuração para Docker e testes automatizados sobre componentes relevantes da aplicação.

Durante o desenvolvimento surgiram também desafios significativos. A modelação do domínio exigiu cuidado, uma vez que várias entidades possuem relações entre si e estados próprios, como sócios, atletas, patrocínios, cobranças e pagamentos. A integração com a Stripe exigiu uma atenção particular ao fluxo de pagamento, à validação de notificações externas e à atualização correta do estado interno da aplicação. Outro desafio foi garantir que a interface se mantinha utilizável apesar da quantidade de funcionalidades disponíveis, sobretudo na área administrativa.

Apesar de a aplicação já cobrir um conjunto alargado de funcionalidades, existem várias melhorias possíveis para trabalho futuro. Entre elas destacam-se o reforço da cobertura de testes, sobretudo em fluxos completos de integração, a melhoria da experiência de utilização em dispositivos móveis nas áreas administrativas, a adição de mecanismos avançados de pesquisa e filtragem, a criação de notificações automáticas para pagamentos ou aprovações pendentes, e a preparação de uma configuração de produção mais completa, incluindo monitorização, cópias de segurança e políticas de segurança adicionais.

Em suma, a JagozApp demonstra a viabilidade de uma plataforma integrada para apoiar a gestão de um clube desportivo local. O projeto permitiu aplicar conhecimentos de desenvolvimento web, arquitetura de software, bases de dados, autenticação, integração com serviços externos, testes e documentação. Mais do que uma aplicação isolada, o resultado final constitui uma base sólida que pode continuar a evoluir de acordo com as necessidades reais do clube.

Referências

- [1] Em Jogo. Em jogo. <https://emjogo.pt/>.
- [2] JetBrains. Kotlin documentation. <https://kotlinlang.org/>.
- [3] Spring. Spring boot. <https://spring.io/projects/spring-boot>.
- [4] PostgreSQL Global Development Group. Postgresql. <https://www.postgresql.org/>.
- [5] Jdbi. Jdbi documentation. <https://jdbi.org/>.
- [6] Stripe. Stripe. <https://stripe.com/pt-pt>.
- [7] ZXing. Zxing: Multi-format 1d/2d barcode image processing library. <https://github.com/zxing/zxing>.
- [8] Meta Open Source. React documentation. <https://react.dev/>.
- [9] Vite. Vite documentation. <https://vite.dev/>.
- [10] Tailwind CSS. Tailwind css documentation. <https://tailwindcss.com/>.
- [11] React Router. React router documentation. <https://reactrouter.com/>.
- [12] i18next. i18next documentation. <https://www.i18next.com/>.

Apêndice A

Exemplo de apêndice

Este é o primeiro parágrafo do apêndice.

Etiam ac leo a risus tristique nonummy. Donec dignissim tincidunt nulla. Vestibulum rhoncus molestie odio. Sed lobortis, justo et pretium lobortis, mauris turpis condimentum augue, nec ultricies nibh arcu pretium enim. Nunc purus neque, placerat id, imperdiet sed, pellentesque nec, nisl. Vestibulum imperdiet neque non sem accumsan laoreet. In hac habitasse platea dictumst. Etiam condimentum facilisis libero. Suspendisse in elit quis nisl aliquam dapibus. Pellentesque auctor sapien. Sed egestas sapien nec lectus. Pellentesque vel dui vel neque bibendum viverra. Aliquam porttitor nisl nec pede. Proin mattis libero vel turpis. Donec rutrum mauris et libero. Proin euismod porta felis. Nam lobortis, metus quis elementum commodo, nunc lectus elementum mauris, eget vulputate ligula tellus eu neque. Vivamus eu dolor.

Nulla in ipsum. Praesent eros nulla, congue vitae, euismod ut, commodo a, wisi. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Aenean nonummy magna non leo. Sed felis erat, ullamcorper in, dictum non, ultricies ut, lectus. Proin vel arcu a odio lobortis euismod. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Proin ut est. Aliquam odio. Pellentesque massa turpis, cursus eu, euismod nec, tempor congue, nulla. Duis viverra gravida mauris. Cras tincidunt. Curabitur eros ligula, varius ut, pulvinar in, cursus faucibus, augue.

Nulla mattis luctus nulla. Duis commodo velit at leo. Aliquam vulputate magna et leo. Nam vestibulum ullamcorper leo. Vestibulum condimentum rutrum mauris. Donec id mauris. Morbi molestie justo et pede. Vivamus eget turpis sed nisl cursus tempor. Curabitur mollis sapien condimentum nunc. In wisi nisl, malesuada at, dignissim sit amet, lobortis in, odio. Aenean consequat arcu a ante. Pellentesque porta elit sit amet orci. Etiam at turpis nec elit ultricies imperdiet. Nulla facilisi. In hac habitasse platea dictumst. Suspendisse viverra aliquam risus. Nullam pede justo, molestie nonummy, scelerisque eu, facilisis vel, arcu.